



Sviluppo guidato dalla specifica (SDD, Spec Driven Development)

Dispensa per chi parte da zero

A chi è rivolta questa dispensa. Questo materiale è pensato per coloro che non hanno mai scritto un software per lavoro, non conoscono il gergo dell'informatica applicata alle aziende e magari stanno studiando matematica, fisica o informatica.

Non serve nessuna esperienza precedente: ogni parola tecnica, la prima volta che compare, viene spiegata in italiano prima di essere riutilizzata nel resto del documento.

Capitolo 0 — Cos'è lo Spec Driven Development

0.1 Un'analogia per cominciare

Immagina di dover far costruire una casa: non ti presenteresti mai da un muratore dicendo solo "fammi una casa bella"...

Prima parleresti con un architetto: quante stanze, quanti piani, dove va la cucina, quanto puoi spendere, quali materiali sono ammessi dal comune.

L'architetto trasforma queste richieste in un disegno tecnico preciso — la **pianta** — che il muratore può seguire senza dover indovinare nulla. Se durante i lavori nasce un dubbio, si torna alla pianta, non si inventa sul momento.

Lo **sviluppo guidato dalla specifica** (in inglese *Spec Driven Development*, abbreviato **SDD**) applica esattamente questa idea alla scrittura di programmi per computer: prima si scrive un documento chiaro — la **specifica** — che descrive che cosa il programma deve fare, con quali regole, quali limiti e come si verifica che sia stato fatto bene; solo dopo si scrive il codice vero e proprio, seguendo quel documento come una pianta.

La parola inglese "spec" è solo l'abbreviazione di *specification*, cioè "specifica", un elenco preciso e verificabile di che cosa deve succedere, scritto prima di cominciare a costruire.

0.2 Perché non si scrive subito il codice?

Chi non ha mai programmato spesso immagina che programmare significhi sedersi al computer e scrivere istruzioni finché il programma funziona.

Nella pratica professionale, però, questo modo di procedere ha un problema serio: se non si è chiarito bene *che cosa* il programma deve fare, si finisce per scrivere codice che fa qualcosa — ma non necessariamente la cosa giusta, o la cosa che serve davvero a chi lo userà.

Il problema si aggrava quando il programma diventa complesso, quando a scriverlo sono più persone insieme, oppure quando il codice deve durare nel tempo ed essere modificato da altri in futuro: in questi casi, avere prima una specifica chiara evita ore di lavoro buttate e discussioni su "ma io pensavo si dovesse fare così".

Oggi questo problema si è fatto ancora più concreto per un motivo nuovo: parte del codice, sempre più spesso, non lo scrive più direttamente una persona, ma un **agente artificiale** (un programma di intelligenza artificiale capace di scrivere codice da solo, seguendo istruzioni).

Un agente IA, se non riceve una specifica chiara, tende a "indovinare" che cosa serve — e indovina spesso male, una specifica ben scritta è quindi il modo più affidabile per guidare sia le persone sia gli agenti artificiali verso lo stesso risultato.

0.3 Da dove nasce questo modo di lavorare

Lo SDD non è un'invenzione isolata, ma il punto di arrivo (per ora) di un percorso lungo quasi trent'anni, fatto di più idee che si sono aggiunte una sull'altra:

- **1996 — Sviluppo guidato dai test** (in inglese *Test Driven Development*, **TDD**). L'informatico Kent Beck, lavorando a un progetto per l'azienda Chrysler, propose un'idea che allora sembrava controintuitiva: scrivere prima un piccolo controllo automatico che verifica se il programma fa la cosa giusta ("test"), e solo dopo scrivere il codice che serve a farlo passare. Il libro che ha reso questa pratica famosa in tutto il mondo è uscito nel 2002.
- **2002 — "Specifica tramite esempi"** (in inglese *Specification by Example*). Un altro informatico molto noto, Martin Fowler, durante un incontro di lavoro coniò questa espressione per descrivere un'idea semplice: il modo migliore per spiegare che cosa deve fare un programma è mostrare esempi concreti di input e di risultato atteso, scritti in un linguaggio comprensibile anche a chi non programma; l'idea è poi diventata un libro molto diffuso, scritto da Gojko Adzic nel 2011.
- **2003-2006 — Sviluppo guidato dal comportamento** (in inglese *Behavior Driven Development*, **BDD**). L'informatico Dan North notò che molte persone, quando iniziavano a scrivere test come da **TDD**, non sapevano bene da dove cominciare né quanto testare.

Propose quindi di descrivere prima il *comportamento* atteso del programma con frasi semplici e leggibili anche da chi si occupa del lato commerciale o organizzativo di un progetto, non solo da chi scrive codice. L'articolo che ha reso pubblica questa idea è del 2006.

- **2025 — "Spec Kit" di GitHub.** GitHub (la piattaforma più usata al mondo per condividere e gestire codice) ha reso pubblico uno strumento gratuito, nato dalla ricerca di un suo dipendente di nome John Lam, che organizza proprio questo modo di lavorare in quattro tappe collegate: **Specifica** → **Piano** → **Compiti** → **Realizzazione**. Lo strumento è stato reso disponibile a tutti nell'autunno del 2025 ed è oggi uno dei modi più diffusi per far scrivere codice a un agente artificiale in modo affidabile.

In sintesi: SDD raccoglie e riordina, attorno a un'unica idea centrale — *la specifica scritta è il punto di riferimento condiviso* — pratiche che il mondo dell'informatica ha già sperimentato e affinato per decenni.

Fonti verificate: repository GitHub Spec Kit (github.com/github/spec-kit); blog ufficiale GitHub sullo Sviluppo Guidato dalla Specifica; voce Wikipedia su *Test-driven development*; articolo originale di Dan North "Introducing BDD" (2006); libro *Specification by Example* di Gojko Adzic (2011).

0.4 Come si avvicina questo metodo partendo da zero, senza sapere nulla di informatica

Non serve sapere programmare per capire l'idea di fondo, bastano quattro passi mentali, nell'ordine:

1. **Scrivi prima che cosa vuoi ottenere, non come lo otterrai.** Anche prima di sapere scrivere una riga di codice, puoi già imparare a descrivere con precisione un obiettivo: che cosa deve succedere, con quali dati in ingresso, con quale risultato atteso, e che cosa invece non deve succedere.
2. **Scomponi il problema in pezzi piccoli e verificabili.** Un obiettivo enorme e vago ("fai un sito che funzioni bene") non si può verificare. Un obiettivo piccolo e preciso ("quando l'utente inserisce un'email senza la @, il programma deve mostrare un messaggio di errore e non salvare nulla") si può verificare in pochi secondi, vero o falso.
3. **Accetta che le domande aperte vanno scritte, non indovinate.** Se non sai una cosa, scrivila come domanda aperta invece di inventare una risposta. È un'abitudine mentale, non una tecnica informatica, ed è probabilmente la più utile di tutte da portarsi dietro in qualunque materia si studi.
4. **Verifica sempre con un criterio oggettivo, non con un'impressione.** "Mi sembra che funzioni" non è una verifica. "Ho controllato che, dato questo input, il risultato è esattamente questo" lo è; questa distinzione è la stessa che si usa in un esperimento di fisica o in una dimostrazione matematica: un'affermazione vale se è verificabile, non se suona convincente.

Il resto di questa dispensa spiega, un passo alla volta e con parole semplici, come questi quattro principi si trasformano in pratica concreta: quali documenti si scrivono, in che ordine, chi li scrive (una persona o un agente artificiale) e come si verifica che il lavoro sia stato fatto bene.

0.5 Un piccolo dizionario dei concetti-chiave, prima di cominciare

Prima di entrare nei dettagli, ecco cinque parole che compariranno spesso, spiegate una volta per tutte con parole comuni:

- **Specifica:** il documento scritto che descrive un obiettivo in modo preciso e verificabile.
- **Agente artificiale** (spesso abbreviato solo "agente"): un programma di intelligenza artificiale capace di leggere una specifica e scrivere codice, oppure di analizzare del codice già scritto e segnalarne i problemi.
- **Verifica:** il controllo oggettivo che dice se un pezzo di lavoro rispetta o no la specifica.
- **Decisione tracciata:** una scelta importante scritta da qualche parte, con la motivazione, invece di restare solo nella testa di chi l'ha presa (e quindi persa non appena quella persona se ne dimentica o cambia progetto).
- **Ruolo:** la parte che una persona o un agente artificiale gioca in un dato momento (chi pianifica, chi scrive il codice, chi controlla, chi cerca punti deboli). Una stessa persona può avere ruoli diversi in momenti diversi, ma **mai lo stesso ruolo di controllo su un lavoro che ha fatto lei stessa:** chi verifica un lavoro non deve essere la stessa persona (o lo stesso agente) che lo ha realizzato, altrimenti il controllo perde valore.

Legenda delle sigle

Qui si utilizzeranno comunque alcune sigle tecniche, perché compaiono davvero nel lavoro reale e prima o poi si incontrano; ogni sigla è spiegata in italiano.

Sigla	Cosa significa per esteso	Spiegazione
SDD	Spec Driven Development	Sviluppo che parte da una specifica scritta e verificabile
SPEC	Specification	Il documento di specifica stesso
ADR	Architectural Decision Record	Foglio che registra una scelta tecnica importante e perché è stata presa
TASK	Task	Un pezzo di lavoro piccolo, delimitato e verificabile
AC	Acceptance Criteria	Le condizioni precise che devono risultare vere perché un lavoro si consideri fatto
DoD	Definition of Done	La definizione condivisa di "davvero finito", non solo "sembra finito"
BDD	Behavior Driven Development	Approccio che descrive il comportamento atteso con frasi semplici e leggibili da tutti
TDD	Test Driven Development	Approccio che scrive prima il controllo automatico e poi il codice
FMEA	Failure Mode and Effects Analysis	Analisi preventiva: quali errori si possono verificare, quanto sono gravi, come evitarli
OODA	Observe, Orient, Decide, Act	Ciclo per decidere in fretta: osserva, orientati, decidi, agisci

Sigla	Cosa significa per esteso	Spiegazione
RAG	Retrieval-Augmented Generation	Generazione di testo che prima recupera informazioni pertinenti e poi le usa per rispondere
SELF - RAG	SELF Retrieval-Augmented Generation	Variante di RAG che lavora sul corpus documentale interno di un progetto
CAG	Context Augmented Generation	Preparazione ordinata del contesto prima di far generare una risposta a un agente
MCP	Model Context Protocol	Un protocollo (un insieme di regole condivise) che permette a un agente artificiale di usare strumenti esterni
LLM	Large Language Model	Un modello di intelligenza artificiale addestrato su grandi quantità di testo, capace di generare linguaggio e codice
HITL	Human In The Loop	Il punto del lavoro in cui deve intervenire per forza il giudizio di una persona
API	Application Programming Interface	L'insieme di regole con cui un programma "parla" con un altro programma
SDK	Software Development Kit	Un pacchetto già pronto di strumenti per collegarsi più facilmente a un servizio
UI	User Interface	La parte del programma che la persona vede e usa
ERP	Enterprise Resource Planning	Un grande programma gestionale che un'azienda usa per contabilità, magazzino, ordini, ecc.
JWT	JSON Web Token	Un piccolo blocco di dati usato per dimostrare che una persona si è autenticata davvero
SQL	Structured Query Language	Il linguaggio con cui si interrogano i database (gli archivi organizzati di dati)
ROI	Return on Investment	Quanto rende, rispetto al costo, un investimento fatto
.NET	—	Una piattaforma della Microsoft per costruire programmi, citata più avanti come esempio concreto

Capitolo 1 — Il metodo

1.1 In cosa consiste il metodo

Nel modo in cui viene usato in questa dispensa, lo Sviluppo Guidato dalla Specifica è un insieme di abitudini di lavoro in cui la specifica scritta, controllabile e sempre aggiornata diventa il punto di riferimento principale per mettere d'accordo tutti — persone e agenti artificiali — su che cosa si sta costruendo, come lo si costruisce, come lo si verifica e come si gestiscono i cambiamenti nel tempo.

Il confine di questa idea non è ancora rigido come quello di **TDD** o **BDD**, che esistono da più tempo: nella pratica di oggi il termine viene usato in senso ampio per indicare qualunque modo di lavorare che metta "prima la specifica" — soprattutto quando il codice viene scritto con l'aiuto di agenti artificiali.

Questa lettura, coerente con l'impostazione recente dello strumento GitHub Spec Kit, si trova all'incrocio tra più discipline già esistenti: l'analisi dei requisiti, la Specifica tramite Esempi, lo Sviluppo Guidato dal Comportamento, lo Sviluppo Guidato dai Test e lo sviluppo assistito da agenti artificiali.

1.2 Da quali idee nasce

Riassumendo quanto già raccontato nel Capitolo 0, l'approccio di oggi mette insieme:

- l'analisi dei requisiti tradizionale, che mette al centro chiarezza, tracciabilità e controllo di che cosa davvero serve;
- la Specifica tramite Esempi, che usa casi concreti come forma di specifica condivisa;
- lo Sviluppo Guidato dal Comportamento, che collega linguaggio comune e comportamento osservabile del programma;
- lo Sviluppo Guidato dai Test, che trasforma i controlli automatici in una guida per scrivere il codice, anziché in un'attività fatta solo alla fine;
- pratiche più vicine al mondo della progettazione rigorosa (contratti espliciti tra le parti di un programma, modelli formali), in cui la correttezza dipende da regole dichiarate in anticipo;
- lo sviluppo recente assistito da agenti artificiali, in cui la qualità del risultato dipende sempre di più dalla qualità del documento di partenza dato in mano all'agente.

La conclusione centrale è: questo metodo non inventa qualcosa di nuovo dal nulla, ma riordina pratiche già esistenti attorno a un'unica regola — *la specifica è il punto di controllo principale tra ciò che si vuole ottenere e ciò che viene realizzato davvero*.

1.3 Quando conviene usarlo

Conviene soprattutto quando è vera almeno una di queste condizioni:

- l'argomento trattato è complesso o regolato da norme precise;
- lavorano insieme più persone, più gruppi o più fornitori esterni;
- un errore costerebbe caro (in denaro, in tempo o in reputazione);
- il programma dovrà vivere a lungo ed essere modificato più volte nel tempo;
- il codice viene scritto anche da agenti artificiali;
- serve poter ricostruire, mesi dopo, perché è stata presa una certa decisione.

Per questo motivo si trova soprattutto in: grandi sistemi aziendali, software di contabilità, ambito legale e finanziario, programmi gestionali di tipo **ERP**, collegamenti tra sistemi diversi tra loro, prodotti soggetti a controlli e verifiche esterne (audit), e in generale ovunque più agenti artificiali debbano comportarsi in modo coerente tra loro.

1.4 I vantaggi, se usato bene

- riduce le incomprensioni tra chi capisce di tecnologia e chi no;

- rende più controllabile il lavoro affidato agli agenti artificiali;
- rende più facile verificare se un risultato è corretto;
- lascia una traccia chiara delle decisioni prese e del perché;
- permette di cambiare strumento o fornitore tecnologico senza perdere la logica di fondo del progetto;
- favorisce il riuso di soluzioni, di pezzi di lavoro e di controlli di qualità già collaudati.

1.5 I rischi, se usato male

- credere che scrivere una specifica basti da sola a garantire la qualità;
- scrivere documenti lunghi, vaghi e impossibili da verificare davvero;
- confondere tra loro tre cose diverse: la specifica di ciò che serve, il piano tecnico e il codice vero e proprio;
- lasciare che la specifica e il codice reale si allontanino nel tempo, finché non si assomiglino più;
- trattare la specifica come un documento decorativo, invece che come la fonte di verità del progetto;
- irrigidire troppo il processo su problemi che invece richiedono ancora di essere esplorati;
- affidare a un agente artificiale compiti troppo grandi rispetto a quanto la specifica sia davvero chiara.

Il rischio più grande, detto semplicemente, nasce quando una specifica **sembra** autorevole senza esserlo davvero.

1.6 La definizione pratica usata in questa dispensa

In questa dispensa, lo Sviluppo Guidato dalla Specifica viene trattato come un piccolo **sistema di lavoro di base** per lo sviluppo di programmi. Il suo nucleo, ridotto all'osso, è composto da: una specifica che fa da fonte di verità, un modo esplicito di prendere decisioni, agenti (persone o programmi di intelligenza artificiale) con ruoli chiari e limitati, alcuni controlli di qualità, costo, rischio e manutenibilità, e una memoria scritta che vive dentro l'archivio del progetto (in inglese *repository*, cioè il luogo dove sono conservati tutti i file di un progetto, comprese le versioni precedenti), leggibile sia dalle persone sia dagli agenti artificiali.

1.7 Un esempio concreto di ambiente di riferimento

In alcuni contesti reali si sceglie una piattaforma di intelligenza artificiale specialistica (nel caso concreto, **Regolo.ai**, gestita dal fornitore Seeweb) come base per certi compiti tematici.

Alcuni programmi assistenti (in questo caso **Codex**, di OpenAI, e **Claude**, di Anthropic), dentro l'editor di scrittura del codice - tipo Visual Studio Code -, restano invece strumenti di lavoro quotidiano: utili per scrivere, controllare e discutere il codice, ma senza lasciare a loro da soli le decisioni più importanti sulla struttura del progetto.

Il lavoro di più agenti coordinati tra loro (in inglese *swarm*, che vuol dire "sciame", come uno sciame di api che lavora insieme) entra in scena solo quando il progetto ne trae davvero vantaggio, non fin da subito.

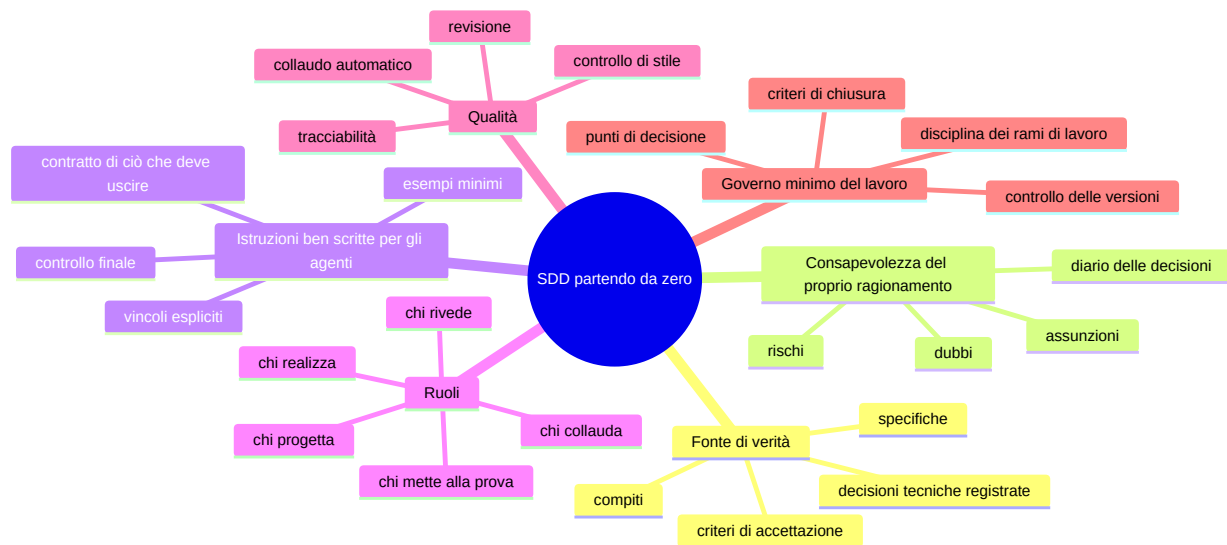
Da qui nascono esigenze concrete: un ambiente dove provare questo metodo su tecnologie come **.NET**, il linguaggio **C#**, **JavaScript**, il database **SQL Server** e gli strumenti di collaudo

automatico; un controllo serio di quanto si spende, di quanto i dati restano riservati e di quanta potenza di calcolo serve e la verifica che il metodo regga anche su un programma gestionale grande e articolato.

La forma più semplice e utile di questo metodo, all'inizio, si riassume in una sola riga:

specifica → piano → compiti → realizzazione → verifica → decisione

Capitolo 2 — Mappa d'insieme del metodo



Capitolo 3 — I principi guida

3.1 La specifica viene prima del codice

Un pezzo di lavoro diventa davvero *affidabile da delegare a un agente artificiale* **solo quando dichiara con chiarezza**: l'obiettivo, il contesto, che cosa entra e che cosa deve uscire, i vincoli, i casi limite e i criteri con cui si stabilisce che è stato fatto bene.

Finché questo non è chiaro, conviene restare nella fase in cui si chiariscono i dubbi (fase di analisi del problema e delle esigenze), invece di correre a scrivere codice.

3.2 L'agente propone, la persona decide

Gli agenti artificiali rendono bene quando esplorano alternative, scrivono bozze, eseguono compiti delimitati, criticano soluzioni già proposte e velocizzano la riscrittura o il collaudo del codice.

Restano invece **fuori dalla loro competenza**:

- decidere scelte di business
- inventare da soli requisiti che mancano
- prendere decisioni difficili dalle quali tornare indietro ** introdurre strutture tecniche complesse solo di propria iniziativa.

3.3 Il lavoro di più agenti insieme richiede materiale condiviso

Un gruppo di agenti IA che lavora bene insieme (uno "sciame", in inglese *swarm*) lavora sempre sugli stessi documenti condivisi: specifiche, elenco dei compiti e criteri con cui si stabilisce che il lavoro è concluso.

Senza questi documenti condivisi, far lavorare più agenti insieme produce soprattutto confusione, non velocità.

3.4 La qualità conta più della quantità

Il metodo sta funzionando quando le decisioni diventano più chiare, si deve rifare meno lavoro, la velocità aumenta senza accumulare problemi nascosti (in gergo, "debito tecnico": lavoro fatto in fretta e male che qualcuno dovrà sistemare più avanti) e il codice resta comprensibile anche dopo settimane o mesi.

3.5 La memoria del progetto deve stare nell'archivio dei file, non nella testa di qualcuno

I documenti di partenza devono avere una sede precisa: file salvati, con versioni tracciate, leggibili e aggiornabili da chiunque lavori al progetto. Una conversazione, per quanto utile nel momento in cui avviene, non può sostituire un documento scritto e conservato.

Capitolo 4 — Da dove cominciare in pratica

4.1 Il minimo indispensabile

Partendo da zero conviene fare subito una distinzione semplice: da una parte c'è il **livello di governo del lavoro**, che si occupa dell'archivio dei file, delle specifiche, delle revisioni e delle decisioni; dall'altra c'è il **livello specialistico**, che lavora sui contenuti specifici di un settore o di un dominio (per esempio: le regole della contabilità, o le regole di un settore legale particolare).

La scelta iniziale più utile è definire bene i ruoli, come in questa tabella:

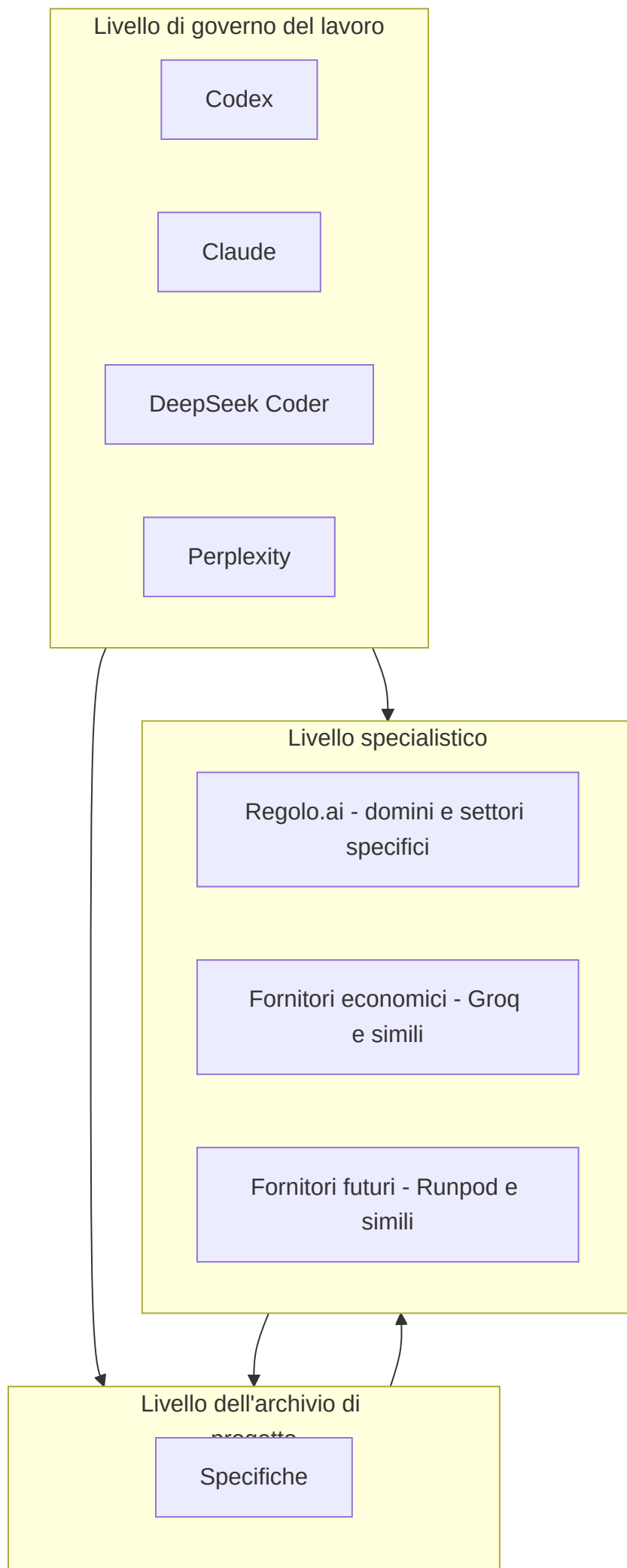
Livello	Cosa serve al minimo	Perché
Editor di scrittura del codice	Un programma come Visual Studio Code	uno spazio di lavoro unico
Controllo delle versioni	Git più un archivio remoto condiviso	poter tracciare ogni modifica e tornare indietro se serve
Governo dell'archivio	Assistenti come Codex e Claude, usati dentro l'editor	pianificazione, scrittura del codice, revisione, governo generale
Aiuto esterno	Altri assistenti dove utile (per esempio DeepSeek Coder e Perplexity)	contenere i costi di scrittura del codice e fare ricerche/verifiche sul web
Collaudo automatico	Lo strumento di test nativo del linguaggio usato	avere una prova oggettiva che il codice funziona

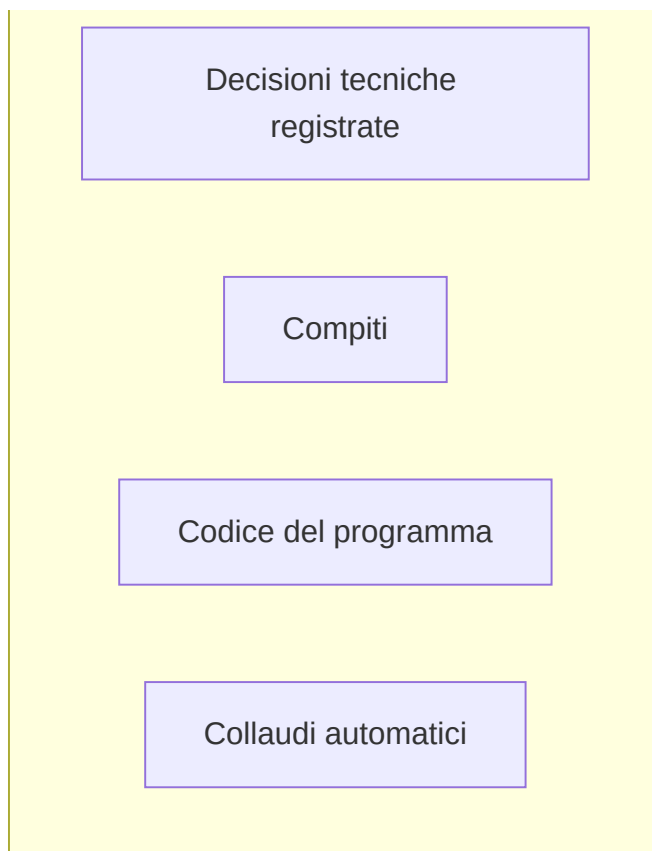
Livello	Cosa serve al minimo	Perché
Specifiche	Una cartella chiamata specs/ con file di testo semplice	il punto di riferimento leggibile da tutti
Decisioni	Una cartella chiamata adr/	il registro delle scelte fatte e del perché
Compiti	Una cartella tasks/ oppure una lavagna semplice di lavoro	dividere il lavoro in pezzi gestibili
Controlli	Collaudo automatico, controllo di stile del codice, verifica finale	i controlli minimi di qualità

All'inizio bastano questi pochi elementi stabili. Strumenti più avanzati — protocolli per collegare gli agenti a strumenti esterni, sistemi di recupero automatico di informazioni, grafi di conoscenza, sistemi di orchestrazione complessi tra molti agenti — avranno senso solo quando coordinare tutto a mano diventerà davvero un ostacolo concreto. Prima di quel momento, aggiungono quasi sempre complicazione e raramente aiuto reale.

4.2 Come si organizzano i livelli di lavoro

Il modo di lavorare acquista chiarezza quando lo si immagina come un sistema a più livelli separati, ciascuno con un compito preciso.





Come si legge questo schema. **Codex**, **Claude**, **DeepSeek** **Coder** e **Perplexity** presidiano il livello di governo del lavoro: gestiscono l'archivio di progetto, le specifiche, la divisione dei compiti, la revisione, la verifica e smistano il lavoro verso gli specialisti giusti. **Regolo.ai** occupa invece il livello primario in cui girano i modelli specialistici di un settore. **Groq** e fornitori simili sono adatti a compiti veloci, economici o molto frequenti, mentre **Runpod** e servizi simili indicano una direzione futura verso maggiore controllo dei costi di calcolo, più libertà di scelta del fornitore ed eventualmente la possibilità di gestire i modelli in autonomia.

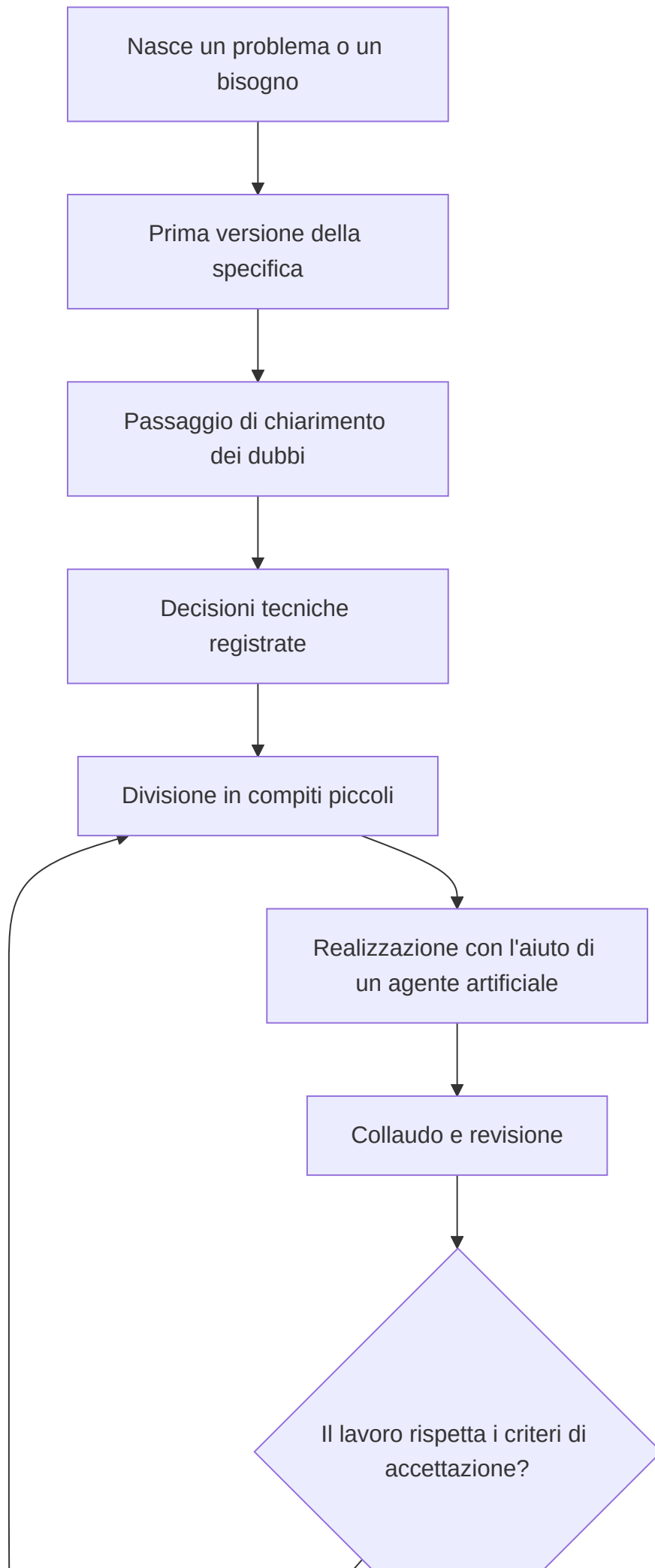
Regola di fondo. Chi governa l'archivio di progetto non entra nel merito dei contenuti specialistici. Chi lavora sui contenuti specialistici non decide il governo dell'archivio di progetto.

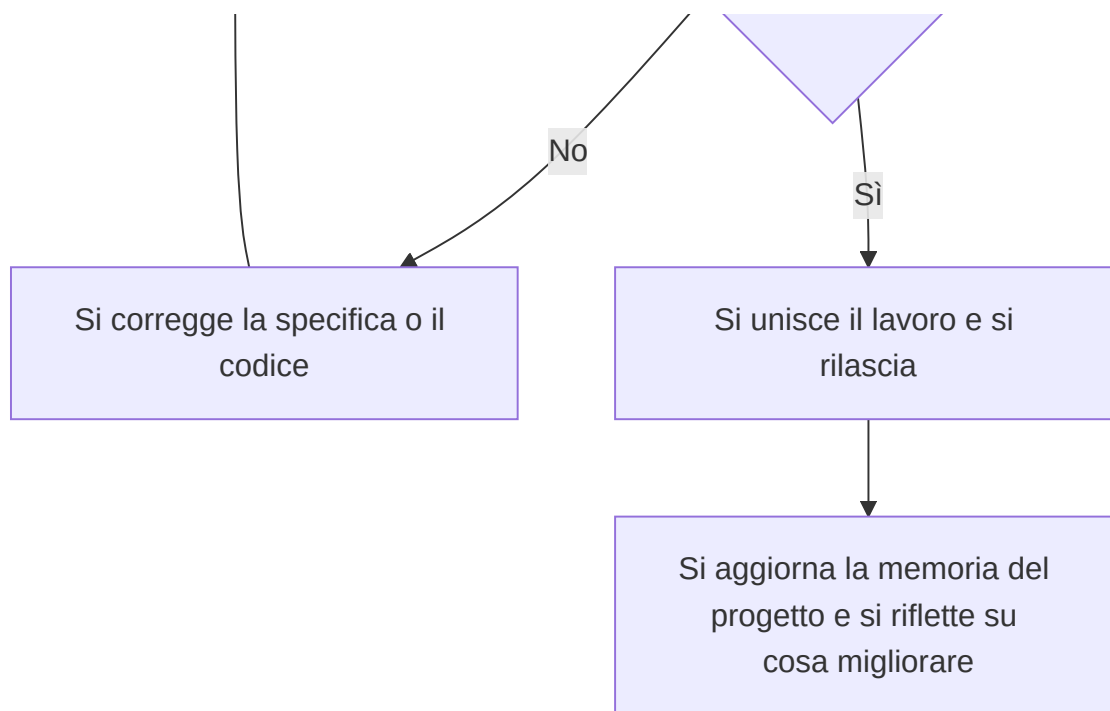
Quando un agente artificiale attraversa entrambi i livelli, serve un accordo scritto e chiaro su che cosa può decidere da solo, che cosa può solo proporre, e dove deve fermarsi.

4.3 Come si organizza l'archivio del progetto

```
cartella-principale-del-progetto/  
  specs/  
    000- costituzione-del-progetto.md  
    001-modulo-autenticazione.md  
    002-modulo-fatturazione.md  
  adr/  
    ADR-001-stile-architettuale.md  
    ADR-002-politica-uso-intelligenza-artificiale.md  
  tasks/  
    TASK-001-avvio-autenticazione.md  
    TASK-002-scadenza-sessione.md  
  prompts/  
    architetto.md  
    realizzatore.md  
    revisore.md  
    collaudatore.md  
    lavagna-condivisa-agenti.md  
  evals/  
    liste-di-controllo-accettazione/  
    casi-di-riferimento/  
  src/  
  tests/  
  docs/  
    diario-delle-decisioni.md  
    registro-delle-assunzioni.md  
    registro-dei-rischi.md
```

4.4 Il ciclo di lavoro di base





Capitolo 5 — Essere consapevoli del proprio ragionamento

5.1 Perché serve

Essere consapevoli del proprio ragionamento, in questo contesto, significa che chi lavora al progetto osserva attivamente il proprio modo di pensare mentre lavora.

In un progetto costruito con questo metodo, questa abitudine serve a restare lucidi nei momenti in cui si rischia di confondere:

- una falsa sensazione di aver capito tutto quando in realtà mancano ancora informazioni;
- il delegare troppo presto a un agente artificiale un compito ancora poco chiaro;
- la confusione tra un fatto verificato, una supposizione e un semplice desiderio;
- l'accumulo di decisioni importanti prese ma mai scritte da nessuna parte.

5.2 Un protocollo minimo, facile da ricordare

Per restare operativi senza complicare troppo il metodo, ogni specifica importante può contenere cinque blocchi ricorrenti, sempre con lo stesso nome:

1. **Fatti noti** — ciò che si sa con certezza, verificato;
2. **Assunzioni** — ciò che si presume vero, ma non è ancora stato verificato;
3. **Domande aperte** — ciò che manca ancora e blocca una decisione definitiva;
4. **Rischi** — ciò che potrebbe andare storto, e quanto sarebbe grave;
5. **Segnali di stop** — condizioni che, se si verificano, devono fermare il lavoro finché una persona non decide che cosa fare.

5.3 Un esempio concreto

Fatti noti

- Il modulo deve gestire l'accesso tramite un token di autenticazione (JWT).
- La tecnologia di riferimento è ASP.NET Core con database SQL Server.

Assunzioni

- Si presume che il token di aggiornamento possa essere conservato lato server.

Domande aperte

- Serve gestire più aziende clienti separate nello stesso sistema?
- È previsto in futuro un accesso unico condiviso con altri servizi (SSO)?

Rischi

- Un token revocato potrebbe restare valido per errore.
- Un utente potrebbe restare agganciato a una sessione non più sua.
- Costi di riscrittura se il modo di gestire gli utenti cambierà in seguito.

Segnali di stop

- Mancano ancora i criteri sulla scadenza della sessione.
- Mancano ancora i casi di errore previsti per le chiamate esterne.

5.4 La regola pratica da ricordare

Un agente artificiale utile separa con chiarezza i fatti dalle assunzioni. Quando questi due piani si mescolano, la qualità del lavoro peggiora molto in fretta — esattamente come, in un esperimento scientifico, confondere un dato misurato con un'ipotesi non ancora verificata porta a conclusioni sbagliate.

Capitolo 6 — Scrivere istruzioni efficaci per un agente artificiale

6.1 Di che cosa si tratta

Scrivere istruzioni efficaci per un agente artificiale (in gergo tecnico, spesso chiamato "prompt engineering", cioè "progettazione delle istruzioni") significa costruire richieste riusabili che mantengano l'agente dentro un perimetro chiaro e lo portino a produrre un risultato controllabile.

In pratica serve a:

- far rispettare all'agente i limiti che gli sono stati dati;
- far dichiarare esplicitamente all'agente ciò che non sa o di cui non è sicuro;
- ottenere un risultato organizzato in modo prevedibile;
- far verificare all'agente il proprio lavoro prima di dire di aver finito.

6.2 Le parti di una buona istruzione

Una buona istruzione operativa, nella maggior parte dei casi, contiene sei parti:

1. il ruolo che l'agente deve assumere;
2. l'obiettivo da raggiungere;
3. il contesto già disponibile;
4. i vincoli da rispettare;
5. la forma esatta in cui deve essere restituito il risultato;
6. le condizioni in cui l'agente deve fermarsi invece di proseguire.

6.3 Un modello di base da riusare

Ruolo:

Sei colui che rivede l'architettura del modulo X.

Obiettivo:

Preparare un piano di realizzazione per la funzionalità Y.

Contesto disponibile:

- Specifica: specs/002-modulo-fatturazione.md
- Decisione tecnica registrata: ADR-002-politica-uso-intelligenza-artificiale.md
- Compito: TASK-014.md

Vincoli:

- Non modificare le regole fiscali già stabilite.
- Non introdurre nuove dipendenze esterne senza motivarlo.
- Se mancano informazioni, elencate e fermati.

Forma del risultato atteso:

1. Sintesi del problema
2. Assunzioni fatte
3. Rischi individuati
4. Piano diviso in passi
5. File che verranno toccati
6. Collaudi automatici da aggiungere

Condizioni di stop:

- se la specifica è ambigua sul calcolo dell'IVA
- se il compito richiede decisioni di business non ancora esplicitate.

6.4 Un'osservazione pratica sui diversi assistenti

Le indicazioni ufficiali più recenti dei principali produttori di questi strumenti convergono su un punto importante:

- per i modelli più recenti di OpenAI, istruzioni brevi e orientate direttamente al risultato finale tendono a funzionare meglio di istruzioni lunghissime e iper-dettagliate;
- per Claude, di Anthropic, le istruzioni funzionano bene quando i criteri di successo sono chiari, la struttura è esplicita e sono previsti controlli intermedi;
- in entrambi i casi, il punto decisivo resta definire bene le prove disponibili, i vincoli e la forma del risultato atteso.

In pratica, conviene lavorare così:

- istruzioni brevi per compiti semplici;
- istruzioni ben strutturate per compiti con molta ambiguità;
- modelli di istruzione già pronti, salvati in una cartella dedicata (prompts/);
- niente dipendenza dalle conversazioni passate come unica memoria del lavoro fatto.

Capitolo 7 — Far lavorare più agenti insieme, partendo da zero

Un gruppo coordinato di agenti (uno "sciame", in inglese *swarm*) si può costruire anche senza una piattaforma dedicata apposita, basta far circolare tra ruoli diversi gli stessi documenti condivisi dell'archivio di progetto e imporre un ordine chiaro di intervento.

La distinzione davvero utile è questa:

- uno sciame che governa l'archivio di progetto;
- uno sciame specialistico che si occupa dei contenuti di un settore.

Tenere questi due gruppi ben distinti evita parecchi equivoci pratici.

7.1 Lo sciame che governa l'archivio di progetto

Questo gruppo governa il ciclo di sviluppo e presidia l'archivio di progetto. Qui conta meno il numero di agenti coinvolti e conta molto di più la chiarezza dei ruoli.

Ruolo	Assistente principale	Funzione
Chi progetta l'architettura	Claude	chiarisce la specifica, i rischi, i confini tecnici
Chi realizza il codice	Codex	modifica il codice, riscrive, collauda, produce risultati concreti
Chi sviluppa con attenzione ai costi	DeepSeek Coder	compiti tecnici molto ripetitivi, a costo controllato
Chi mette alla prova con ricerche esterne	Perplexity	verifica esterna, alternative, ricerca, contraddittorio

7.2 Lo sciame specialistico di un settore

Questo gruppo vive prevalentemente su una piattaforma dedicata (nell'esempio, [Regolo.ai](#)) e, se serve, si appoggia a fornitori economici o dedicati.

Qui il centro dell'attenzione si sposta dal governo dell'archivio alla correttezza dei contenuti di un settore specifico.

Ruoli tipici:

- esperto fiscale;

- esperto di sistemi gestionali;
- specialista di sicurezza informatica;
- specialista di qualità;
- specialista del modo in cui sono organizzati i dati;
- specialista di collegamenti tra sistemi diversi.

La regola di confine resta semplice:

- il livello di governo decide *chi interpellare*;
- il livello specialistico decide *che cosa è corretto nel proprio settore*.

7.3 Un percorso di crescita consigliato

Livello 0 — Un solo agente. In questa fase un solo agente basta per: esplorare il codice esistente, realizzare compiti piccoli, scrivere collaudi automatici, spiegare una riscrittura.

Livello 1 — Due agenti. Entrano in scena due ruoli distinti: chi progetta e pianifica, chi realizza e poi rivede. Un esempio semplice: Claude aiuta a chiarire la specifica e i rischi, Codex realizza e rifinisce il codice, e infine uno dei due (o entrambi) fa una revisione critica finale.

Se il quadro si allarga: Claude o Codex governano il compito dal lato dell'archivio di progetto, DeepSeek Coder assorbe parte del carico tecnico a costo ridotto, Perplexity fa da contraddittorio esterno sulle assunzioni fatte.

Livello 2 — Uno sciame delimitato. Quattro ruoli stabili: chi progetta, chi realizza, chi rivede, chi collauda e mette alla prova.

Livello 3 — Uno sciame più ampio. Ruoli aggiuntivi solo se servono davvero: chi rivede la sicurezza, chi rivede le prestazioni, chi mette alla prova un settore specifico, chi si occupa della documentazione.

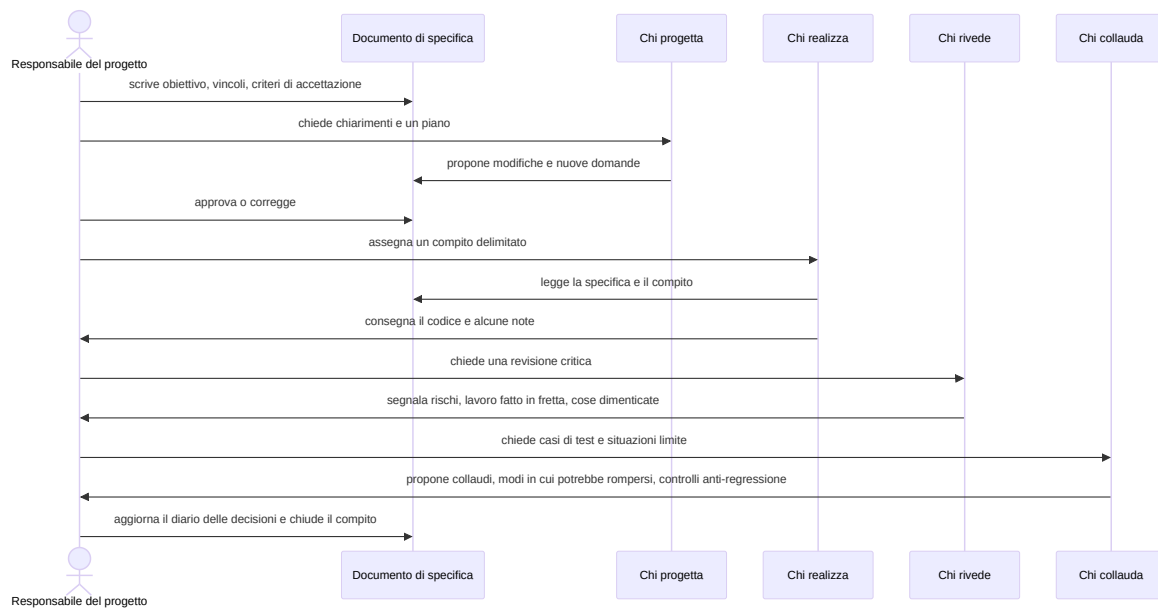
7.4 Quando aggiungere un nuovo ruolo

Un nuovo agente entra in gioco quando sono vere tutte e tre queste condizioni:

- esiste un tipo di lavoro che si ripete spesso;
- quel ruolo ha input e output stabili e prevedibili;
- il valore che porta supera il costo di doverlo coordinare.

Lo stesso criterio vale per scegliere un nuovo fornitore tecnologico: conviene introdurne uno nuovo solo quando migliora davvero almeno uno tra: qualità del risultato, costo, velocità di risposta o riservatezza dei dati.

7.5 Come interagiscono i ruoli, passo per passo



7.6 I sei cappelli per pensare, applicati al lavoro di gruppo

I "sei cappelli per pensare" sono un metodo, ideato dallo psicologo Edward de Bono, per affrontare un problema guardandolo di volta in volta da un solo punto di vista alla volta, invece di mescolare tutto insieme in una discussione confusa.

Applicato al lavoro con più agenti, aiuta a far emergere in ordine: dati, rischi, alternative, valore e decisione finale.

Cappello (colore)	Domanda guida	Come si usa in questo metodo
Bianco (i fatti)	Che cosa sappiamo davvero?	fatti, dati in ingresso, collegamenti, vincoli
Rosso (le sensazioni)	Che cosa ci preoccupa o ci attira?	intuizioni, disagi, segnali di rischio non ancora formalizzati
Nero (le criticità)	Che cosa può andare storto?	sicurezza, costi, manutenzione, dipendenza da un solo fornitore
Giallo (le opportunità)	Dove sta il valore?	produttività, riduzione dell'ambiguità, ritorno dell'investimento
Verde (la creatività)	Quali alternative esistono?	soluzioni diverse, strumenti diversi, modi diversi di procedere
Blu (la decisione)	Come governiamo il processo?	ordine del giorno, punti di controllo, decisione finale

Un piccolo rituale pratico da 20 minuti. Su un compito non banale, basta già una sequenza breve come questa:

- prima il cappello Bianco su specifica e fatti;
- poi il cappello Nero su rischi e possibili errori;
- poi il cappello Verde su due o tre alternative;
- poi il cappello Giallo sul valore della soluzione scelta;
- infine il cappello Blu su decisione, responsabile e prossimo passo.

Il cappello Rosso entra bene quando emergono disagi o intuizioni che ancora non hanno trovato una forma tecnica precisa.

Capitolo 8 — Alcuni schemi per decidere meglio (frameworks decisionali)

8.1 Cynefin

Questo schema (si legge più o meno "kevinin", di origine gallese) aiuta a capire quanto ci si possa fidare di delegare un compito a un agente artificiale, in base a quanto il problema è conosciuto:

- **Chiaro:** compito ripetibile, si può delegare con fiducia a un assistente come Codex o Claude;
- **Complicato:** serve un confronto tra alternative e una doppia revisione;
- **Complesso:** serve esplorazione, prove preliminari, prototipi, un piccolo gruppo di agenti delimitato;
- **Caotico:** prima si limita il danno, solo dopo si prova a capire lo schema del problema.

8.2 OODA

Per organizzare il lavoro settimana per settimana, resta un riferimento molto pratico, in quattro fasi:

- **Osserva** (leggi i segnali da errori, collaudi, riscontri);
- **Orientati** (rimetti ordine nella specifica e nei vincoli);
- **Decidi** (scegli il prossimo pezzo di lavoro da affrontare);
- **Agisci** (realizza e verifica).

8.3 FMEA — analisi preventiva dei guasti

Su parti particolarmente delicate del programma (accesso degli utenti, autorizzazioni, gestione dei pagamenti, notifiche, importazione ed esportazione dei dati) diventa molto utile chiedersi in anticipo:

- quale errore si può verificare?
- Quanto sarebbe grave?
- Quanto è probabile che succeda?
- Quanto sarebbe facile accorgersene?
- Che cosa si può fare in anticipo per evitarlo?

8.4 Kepner-Tregoe

Questo schema funziona bene quando bisogna confrontare tra loro alternative tecniche diverse, per esempio: un modello di intelligenza artificiale molto potente contro uno più leggero e aperto; lavorare su un servizio esterno (in gergo, "nel cloud") contro lavorare su un computer locale; agenti separati contro un solo agente che fa tutto; un programma unico contro tanti piccoli programmi collegati tra loro.

Capitolo 9 — Alcuni schemi di progettazione da preferire (e alcuni errori da evitare)

9.1 Schemi legati al modo di lavorare

- Specifica → Piano → Compiti → Realizzazione → Verifica;
- una decisione tecnica registrata per ogni scelta importante;
- una definizione esplicita di "davvero finito";
- una revisione fatta rispettando un accordo scritto in anticipo, non a sensazione.

9.2 Schemi legati al lavoro con gli agenti artificiali

- chi pianifica, chi realizza, chi rivede, come ruoli distinti;
- un ciclo di valutazione e miglioramento continuo del risultato;
- un ciclo di critica che rilegge il proprio stesso lavoro prima di consegnarlo;
- uno smistatore che indirizza ogni compito verso il tipo di agente più adatto;
- una "lavagna condivisa" fatta semplicemente di file di testo leggibili da tutti gli agenti coinvolti.

9.3 Schemi legati alla struttura del programma

Questi sono nomi tecnici che si incontrano spesso, spiegati in una riga ciascuno:

- **Architettura a nucleo protetto** (in inglese *hexagonal architecture*): il cuore del programma resta isolato dal mondo esterno, e ogni collegamento verso l'esterno passa da un punto di ingresso ben definito, così cambiare un servizio esterno non obbliga a riscrivere tutto il resto.
- **Area delimitata di competenza** (in inglese *bounded context*): una parte del programma che si occupa di un solo argomento, con le sue regole precise, senza mescolarsi con le regole di altre parti.
- **Strato di protezione tra sistemi diversi** (in inglese *anti-corruption layer*): un pezzo di codice che traduce i dati tra due sistemi con logiche diverse, così un sistema non "sporca" le regole dell'altro.
- **Modo standard di accedere ai dati salvati** (in inglese *repository pattern*): un solo punto del codice che sa come leggere e scrivere i dati, così il resto del programma non deve saperlo.
- **Test del contratto tra due parti** (in inglese *contract tests*): controlli automatici che verificano che due parti di un programma continuino a "capirsi" allo stesso modo anche dopo una modifica.
- **Interruttori per attivare o disattivare una funzione** (in inglese *feature flags*): un modo per accendere o spegnere una parte del programma senza dover riscrivere il codice, utile per provare qualcosa con prudenza.

9.4 Errori da evitare

- un'unica istruzione enorme per governare tutto il progetto;
- agenti senza ruoli chiari e senza condizioni precise per fermarsi;
- codice scritto senza nessuna specifica alle spalle;

- una revisione fatta dallo stesso agente che ha realizzato il lavoro, senza un vero contraddittorio;
- dipendere dalla memoria delle conversazioni passate invece che dai documenti scritti;
- introdurre troppo presto strumenti di coordinamento complessi che non servono ancora davvero.

Capitolo 10 — I quattro file che contano di più all'inizio

Per partire da zero bastano pochi file tenuti bene, spesso più utili di molti strumenti aggiunti troppo presto.

docs/diario-delle-decisioni.md

Contiene, per ogni decisione importante: che cosa è stato deciso; perché; quali alternative sono state scartate; quale effetto ci si aspetta; la data; chi ha deciso.

docs/registro-delle-assunzioni.md

Contiene, per ogni assunzione: che cosa si presume vero; quanto ci si fida di questa presunzione; quale prova manca ancora; come si intende verificarla.

docs/registro-dei-rischi.md

Contiene, per ogni rischio: di che rischio si tratta; quanto sarebbe grave; quanto è probabile; come lo si previene o attenua; a partire da quale segnale bisogna far intervenire una persona.

prompts/lavagna-condivisa-agenti.md

Contiene il modo standard con cui si avvia una discussione tra più ruoli diversi (agenti o persone), così da non doverlo reinventare ogni volta.

Capitolo 11 — Un esempio di percorso su una piattaforma specialistica (Regolo.ai)

Se una piattaforma specialistica come **Regolo.ai** viene scelta come asse portante del metodo, il percorso più solido è graduale:

- fase 1: **Regolo.ai** come luogo dei modelli specialistici e come vocabolario comune di lavoro;
- fase 2: **Regolo.ai** come raccolta organizzata di ruoli di settore, istruzioni pronte, liste di controllo e regole di smistamento;
- fase 3: **Regolo.ai** come punto di passaggio principale per tutto il livello specialistico;
- fase 4: un'eventuale integrazione più stretta tra chi governa l'archivio di progetto e gli specialisti di settore.

La logica, detta in modo semplice, è: prima si consolidano ruoli e accordi chiari, solo dopo si automatizza tutto.

11.1 Una distinzione da tenere sempre a mente

Il governo dell'archivio di progetto resta più leggibile e controllabile quando è affidato ad assistenti come **Codex**, **Claude**, **DeepSeek** **Coder** e **Perplexity**.

Regolo.ai diventa invece il luogo naturale in cui far vivere:

- i modelli specialistici di un settore,
- le applicazioni verticali su un singolo tema,
- gli agenti dedicati a un argomento specifico,
- le regole con cui interrogare i modelli legati al business.

11.2 Come smistare il lavoro

Tipo di lavoro	Dove va principalmente	Alternativa
Pianificazione dell'archivio e divisione in compiti	Claude o Codex	DeepSeek Coder
Scrittura e riscrittura del codice	Codex	DeepSeek Coder
Revisione critica e chiarimento dei dubbi	Claude	Codex
Ricerca sul web e messa alla prova esterna	Perplexity	Claude
Contenuti di un settore o di un'attività specifica	Regolo.ai	fornitore specialistico esterno
Compiti veloci, economici, poco delicati	Groq o simili	DeepSeek Coder
Carichi di lavoro portabili o semi-dedicati	Runpod o simili	fornitore gestito esterno

11.3 Una strategia semplice su costo e riservatezza

1. per codice proprietario delicato: usare assistenti e specialisti con regole prudenti e compiti ridotti al minimo necessario;
2. per compiti tecnici ripetitivi e poco delicati: spostarli dove costano meno;
3. per compiti di settore molto delicati: preferire il livello specialistico principale, anche se costa di più;
4. per una crescita futura: valutare fornitori più controllabili (come Runpod) solo quando il volume di lavoro giustifica davvero il cambio.

Capitolo 12 — Un percorso a 30, 60, 90 giorni

Primi 30 giorni

- scegliere un piccolo progetto pilota, ma reale;
- creare le cartelle **specs/**, **adr/**, **tasks/**, **prompts/**, **docs/**;

- scrivere la costituzione del progetto (le regole di fondo);
- scegliere i quattro assistenti principali in base al ruolo, non alla moda del momento;
- lavorare con un solo agente o con due agenti governati da Codex e Claude;
- misurare tempo impiegato, errori trovati, quanto lavoro va rifatto, qualità percepita.

Dai 31 ai 60 giorni

- introdurre un piccolo gruppo di agenti delimitato su compiti non banali;
- stabilizzare i ruoli di chi progetta, chi realizza, chi rivede, chi collauda;
- introdurre DeepSeek Coder per assorbire lavoro tecnico a costo contenuto;
- introdurre Perplexity come voce esterna che mette alla prova le assunzioni;
- creare liste di controllo per la revisione;
- introdurre valutazioni manuali o semi-automatiche su casi di riferimento già noti;
- confrontare costo e qualità tra modelli diversi.

Dai 61 ai 90 giorni

- individuare i compiti più adatti a essere delegati del tutto;
- definire regole chiare su dati delicati e codice proprietario;
- spostare i contenuti specialistici sul livello dedicato (per esempio Regolo.ai);
- usare Groq e fornitori simili per picchi di lavoro economici e a basso rischio;
- introdurre agenti dedicati solo sui lavori davvero ricorrenti;
- decidere se serve davvero uno strumento più avanzato (protocolli di collegamento, orchestrazione, livelli di conoscenza condivisa, modelli locali o fornitori dedicati).



Capitolo 13 — Che cosa insegna l'esperienza raccolta finora

Dall'insieme delle prove fatte finora emerge un insegnamento piuttosto chiaro: quando manca una prova solida, i migliori agenti devono convergere su una chiusura prudente, non su un'invenzione plausibile.

Un buon gruppo di lavoro (persone e agenti insieme) non "riempie i buchi" della conoscenza mancante: li rende visibili: il valore di lavorare in gruppo sta proprio nel rendere più chiari i confini, i rischi e le alternative disponibili.

Da qui nasce una regola di fondo, valida anche fuori dall'informatica:

se una specifica non regge il confronto critico di un gruppo di persone preparate, non reggerà nemmeno il passaggio alla realizzazione pratica.

Capitolo 14 — Come scegliere gli strumenti senza farsi guidare dalle mode

La domanda di partenza più utile raramente è quella più affascinante. Conta molto di più chiedersi: *qual è lo strumento più adatto a questo tipo di lavoro, a questo costo e con questo livello di rischio?*

14.1 Una griglia di valutazione semplice

Criterio	Domanda da farsi
Scrittura del codice	Scrive bene nel linguaggio e nella tecnologia scelti?
Riscrittura del codice	Mantiene la struttura e i vincoli già esistenti?
Collaudo	Genera casi di test utili o solo superficiali?
Revisione	Trova problemi reali o fa solo commenti generici?
Gestione del contesto	Regge specifiche lunghe e archivi di progetto grandi?
Costo	Il valore ottenuto supera il costo effettivo?
Riservatezza	Dove passano il codice e i dati usati?
Facilità di guida	Si lascia guidare bene da specifiche e istruzioni chiare?

14.2 Un possibile insieme di strumenti

- Claude come assistente forte per chiarire, progettare, rivedere e valutare i rischi;
- Codex come assistente forte per la realizzazione tecnica dentro l'archivio di progetto;
- DeepSeek Coder come assistente tecnico più economico per compiti di scrittura del codice ben scelti;
- Perplexity come assistente di ricerca, contesto esterno e messa alla prova critica;
- Regolo.ai come casa principale degli specialisti di settore;
- Groq e simili come opzione economica per compiti veloci a basso rischio;
- Runpod e simili come direzione futura per controllare meglio i costi di calcolo e la libertà di scelta del fornitore.

Capitolo 15 — Tabella riassuntiva di ruoli, strumenti e limiti

Questa tabella definisce l'accordo minimo tra strumenti, ruoli, ciò che ricevono in ingresso, ciò che restituiscono, i loro limiti e dove operano. Va considerata come punto di partenza da aggiornare, non come elenco definitivo.

Strumento o gruppo	Ruolo principale	Cosa riceve	Cosa restituisce	Limiti principali	Dove opera
Codex	realizzatore nell'archivio di progetto	specifica, compito, file da modificare, vincoli tecnici	codice modificato, collaudi, note sul lavoro fatto	rischio di procedere troppo senza un confronto critico esterno	governo dell'archivio

Strumento o gruppo	Ruolo principale	Cosa riceve	Cosa restituisce	Limiti principali	Dove opera
Claude	progettista e revisore nell'archivio di progetto	specifica, decisioni tecniche, compiti, rischi, contesto	chiarimenti, piani, revisioni, rischi residui	meno conveniente per compiti enormi e molto ripetitivi	governo dell'archivio
DeepSeek Coder	sviluppatore attento ai costi	compiti di scrittura del codice ben delimitati	correzioni, riscritture meccaniche, impalcatura dei collaudi	qualità non costante su compiti molto ambigui o delicati	governo dell'archivio, fascia economica
Perplexity	chi mette alla prova con ricerche esterne	domanda da verificare, affermazione da controllare, criteri di confronto	fonti, alternative, obiezioni, controargomentazioni	poco adatto a scrivere codice proprietario complesso	governo dell'archivio, ricerca
Specialisti su Regolo.ai	livello specialistico di settore	specifiche di settore, regole del settore, casi reali	risposte specialistiche, regole applicate, decisioni di settore	richiede una buona definizione dei ruoli specialistici	livello specialistico principale
Groq e simili	compiti veloci ed economici	compiti rapidi, poco delicati, molto frequenti	risposte rapide ed economiche, confronti tecnici	contesto limitato, stabilità e riservatezza da valutare caso per caso	livello specialistico, uso opportunistico
Runpod e simili	calcolo portatile	carichi di lavoro ripetibili, modelli da controllare direttamente	calcolo dedicato o semi-dedicato	maggiore complessità di gestione	livello specialistico, direzione futura

Capitolo 16 — Dieci esempi pratici, in ambito contabile, legale e finanziario

Questa sezione mostra dieci esempi concreti di applicazione del metodo, pensati per strumenti interni, applicazioni web o settori verticali di software.

Ogni esempio segue la stessa struttura fissa: contesto e tipo di prodotto; obiettivo; specifica minima; primo gruppo di compiti; criteri di accettazione; gruppo di agenti consigliato; rischi principali.

Chi legge senza alcuna esperienza informatica può trattare questi esempi come dieci piccoli casi di studio: non serve capire ogni dettaglio tecnico, ma è utile osservare come lo stesso schema di ragionamento si ripeta identico su argomenti molto diversi tra loro.

16.1. Strumento contabile per la riconciliazione bancaria

- **Tipo:** strumento interno
- **Obiettivo:** abbinare automaticamente i movimenti di un conto bancario con le registrazioni contabili, riducendo il lavoro manuale di confronto
- **Specifica minima:** importazione dell'estratto conto (file CSV o XLSX); importazione delle scritture contabili; regole automatiche di abbinamento; una coda per i casi dubbi
- **Primo gruppo di compiti:** lettura del file in ingresso; motore che confronta e abbina i dati secondo regole; una schermata che mostra le eccezioni; esportazione delle riconciliazioni fatte
- **Criteri di accettazione:** ogni abbinamento deve essere tracciabile; deve essere sempre chiaro se un abbinamento è automatico o manuale; deve esistere un registro di controllo esportabile
- **Gruppo di agenti consigliato:** Claude per specifica e rischio contabile; Codex per il motore di confronto; DeepSeek Coder per riscritture e collaudi su tabelle; uno specialista contabile su Regolo.ai per le regole di settore
- **Rischi principali:** una riconciliazione fatta per errore; un collegamento sbagliato nel piano dei conti; ambiguità nei movimenti raggruppati insieme

16.2. Applicazione web per la gestione delle fatture da pagare

- **Tipo:** applicazione web
- **Obiettivo:** gestire ricezione, controllo, approvazione e registrazione contabile delle fatture ricevute dai fornitori
- **Specifica minima:** anagrafica fornitori; caricamento delle fatture; percorso di approvazione; esportazione verso il sistema gestionale
- **Primo gruppo di compiti:** ricezione del documento; stato della fattura; regole di approvazione; collegamento di esportazione verso il gestionale
- **Criteri di accettazione:** lo stato di ogni fattura deve essere sempre determinato con certezza; ogni rifiuto deve avere una motivazione obbligatoria; l'esportazione non deve mai duplicare i dati
- **Gruppo di agenti consigliato:** Claude per il percorso di lavoro; Codex per la parte tecnica e i collegamenti; Perplexity per confrontare processi simili già esistenti; uno specialista del ciclo fornitori su Regolo.ai
- **Rischi principali:** doppia registrazione contabile; un percorso di approvazione non verificabile; forte dipendenza da un solo sistema gestionale

16.3. Strumento verticale per le chiusure contabili mensili

- **Tipo:** applicazione specialistica di settore
- **Obiettivo:** coordinare liste di controllo, ratei, risconti, quadrature e approvazioni per la chiusura mensile dei conti
- **Specifica minima:** calendario delle scadenze di chiusura; lista di controllo per ogni area; dipendenze tra i vari compiti; prove del completamento di ogni passo
- **Primo gruppo di compiti:** modello della lista di controllo; gestione delle dipendenze tra compiti; avvisi sulle scadenze; una schermata di avanzamento
- **Criteri di accettazione:** un compito dipendente non può partire senza autorizzazione; ogni passo ha un responsabile esplicito; le prove devono essere scaricabili
- **Gruppo di agenti consigliato:** Claude per le dipendenze del processo; Codex per il motore che gestisce il flusso di lavoro; DeepSeek Coder per la parte di gestione amministrativa; uno specialista delle chiusure contabili su Regolo.ai
- **Rischi principali:** chiusure fatte fuori sequenza; compiti segnati come completati senza prova reale; un flusso di lavoro reso troppo rigido

16.4. Strumento legale per la gestione delle clausole contrattuali

- **Tipo:** strumento interno
- **Obiettivo:** classificare le clausole dei contratti, evidenziarne le varianti e mantenere un catalogo riusabile
- **Specifica minima:** archivio di modelli contrattuali; etichettatura delle clausole; confronto tra versioni diverse; avviso sulle clausole più delicate
- **Primo gruppo di compiti:** archivio delle clausole; motore di confronto tra versioni; libreria di etichette; rapporto sulle differenze trovate
- **Criteri di accettazione:** ogni clausola ha una fonte e una versione tracciata; il confronto tra versioni è leggibile; le modifiche più rischiose generano un avviso
- **Gruppo di agenti consigliato:** Claude per la struttura documentale; Perplexity per confrontare con prassi pubbliche già note; Codex per il motore di confronto e il catalogo; uno specialista di diritto contrattuale su Regolo.ai
- **Rischi principali:** affermazioni legali eccessive rispetto a ciò che è davvero verificato; una falsa equivalenza tra clausole solo apparentemente simili; assenza di tracciabilità tra le versioni

16.5. Applicazione web per l'accoglienza di nuove pratiche legali

- **Tipo:** applicazione web
- **Obiettivo:** raccogliere le richieste, classificarne l'urgenza, raccogliere i documenti e smistare la pratica alla persona giusta
- **Specifica minima:** modulo di raccolta della richiesta; classificazione del tipo di pratica; raccolta degli allegati; assegnazione al professionista competente
- **Primo gruppo di compiti:** modulo con controlli di validità; una casella per le nuove pratiche; assegnazione; cronologia degli stati
- **Criteri di accettazione:** nessuna pratica priva di classificazione; ogni allegato tracciato; il tempo massimo di risposta sempre visibile
- **Gruppo di agenti consigliato:** Claude per modellare il percorso; Codex per l'applicazione e l'archiviazione; Perplexity per confrontare standard simili già noti; uno specialista dell'area legale su Regolo.ai

- **Rischi principali:** una raccolta di informazioni incompleta; dati delicati gestiti male; una prima classificazione imprecisa

16.6. Strumento legale per il monitoraggio di scadenze e adempimenti

- **Tipo:** applicazione specialistica di settore
- **Obiettivo:** tenere sotto controllo scadenze contrattuali, societarie e procedurali, con prove e avvisi
- **Specifica minima:** calendario degli adempimenti; regole di preavviso; assegnazione dei responsabili; prova dell'adempimento avvenuto
- **Primo gruppo di compiti:** motore delle scadenze; promemoria; assegnazioni; rapporto storico
- **Criteri di accettazione:** ogni scadenza ha un responsabile e uno stato; i promemoria sono configurabili; lo storico non può essere alterato
- **Gruppo di agenti consigliato:** Claude per la struttura di governo; Codex per il motore di pianificazione; DeepSeek Coder per le parti ripetitive; uno specialista di conformità normativa su Regolo.ai
- **Rischi principali:** promemoria troppo frequenti e ignorati; scadenze duplicate; una falsa sensazione di essere in regola

16.7. Strumento finanziario per la previsione dei flussi di cassa

- **Tipo:** strumento decisionale
- **Obiettivo:** stimare i flussi di cassa a 30, 60 e 90 giorni, con più scenari possibili
- **Specifica minima:** dati in ingresso su saldi, incassi, pagamenti e scadenze; scenari di base, prudente e aggressivo; spiegazione delle assunzioni fatte
- **Primo gruppo di compiti:** modello dei dati di cassa; motore degli scenari; grafici; rapporto sulle assunzioni
- **Criteri di accettazione:** ogni scenario mostra le proprie assunzioni; è sempre chiara la differenza tra dato osservato e ipotesi; il risultato è esportabile
- **Gruppo di agenti consigliato:** Claude per chiarire le assunzioni; Codex per il motore degli scenari; Perplexity per confrontare pratiche di previsione già note; uno specialista di tesoreria su Regolo.ai
- **Rischi principali:** confondere una previsione con un dato consuntivo già avvenuto; scenari non spiegabili con chiarezza; dipendenza da dati di partenza poco puliti

16.8. Applicazione web per la valutazione del rischio cliente

- **Tipo:** applicazione web
- **Obiettivo:** riassumere il profilo di un cliente, la sua esposizione, l'affidabilità e i segnali di attenzione
- **Specifica minima:** anagrafica del cliente; indicatori di rischio; regole di valutazione; spiegazione del punteggio ottenuto
- **Primo gruppo di compiti:** raccolta dei dati; scheda di valutazione configurabile; schermata del rischio; registro delle decisioni
- **Criteri di accettazione:** il punteggio è sempre spiegabile; le variazioni nel tempo sono tracciate; nessuna decisione automatica avviene senza una prova a supporto

- **Gruppo di agenti consigliato:** Claude per le regole e il governo generale; Codex per il motore di valutazione; DeepSeek Coder per rapporti e schermate; uno specialista di rischio di credito su Regolo.ai
- **Rischi principali:** un punteggio che nessuno sa più spiegare; distorsioni presenti nei dati di partenza; un uso improprio del punteggio ottenuto

16.9. Strumento finanziario per prezzi e margini

- **Tipo:** applicazione specialistica di settore
- **Obiettivo:** calcolare prezzi, costi, margini e analisi di sensibilità per linee di prodotto o servizio
- **Specifica minima:** costi diretti e indiretti; regole di ripartizione dei costi; scenari di prezzo; vista sulla marginalità
- **Primo gruppo di compiti:** modello dei costi; simulatore dei prezzi; motore di sensibilità; rapporto sulla marginalità
- **Criteri di accettazione:** ogni formula è tracciabile; è chiara la differenza tra costo osservato e ipotesi; gli scenari sono confrontabili tra loro
- **Gruppo di agenti consigliato:** Claude per la struttura del modello; Codex per calcoli e rapporti; Perplexity per alternative metodologiche; uno specialista di controllo di gestione su Regolo.ai
- **Rischi principali:** formule poco trasparenti; ripartizioni dei costi arbitrarie; simulazioni scambiate per verità certe

16.10. Strumento trasversale per la verifica documentale preventiva (due diligence)

- **Tipo:** strumento e applicazione web insieme
- **Obiettivo:** raccogliere documenti, classificarli, segnalare le mancanze e produrre una lista di cose da seguire
- **Specifica minima:** archivio documentale condiviso; lista di controllo per categoria; stato di ogni documento; registro delle mancanze
- **Primo gruppo di compiti:** raccolta dei documenti; classificazione; stato della lista di controllo; esportazione del rapporto sulle mancanze
- **Criteri di accettazione:** ogni documento ha categoria, stato e responsabile; il registro delle mancanze è esportabile; lo storico delle attività non è modificabile
- **Gruppo di agenti consigliato:** Claude per la struttura della lista di controllo; Codex per il portale e il flusso di lavoro; Perplexity per confrontare liste di controllo pubbliche già note; specialisti legale, contabile e finanziario su Regolo.ai
- **Rischi principali:** una classificazione troppo generica; documenti segnati come completi senza esserlo davvero; responsabilità che si sovrappongono tra più persone

Capitolo 17 — Modelli di istruzioni permanenti per ogni ruolo (AGENTS.md)

I blocchi seguenti sono modelli di partenza per un file che si chiama, per convenzione, **AGENTS.md** (o una sezione equivalente), da collocare nell'archivio di progetto.

Sono scritti in modo asciutto e diretto, pensati per essere riusati così come sono.

AGENTS.md — Codex

AGENTS.md — Codex

Ruolo

Realizzatore principale nell'archivio di progetto.

Obiettivi

- tradurre compiti delimitati in modifiche concrete al codice;
- aggiungere collaudi automatici e controlli minimi;
- restare coerente con la specifica e le decisioni tecniche registrate.

Che cosa deve ricevere prima di iniziare

- il file di specifica;
- il compito assegnato;
- i file da modificare;
- i vincoli tecnici;
- i criteri di accettazione.

Che cosa deve restituire

- l'elenco dei file toccati;
- la modifica realizzata;
- i collaudi aggiunti o quelli ancora mancanti;
- i rischi rimasti aperti;
- le assunzioni fatte, dichiarate con chiarezza.

Cosa non deve mai fare

- non deve inventare requisiti di business;
- non deve introdurre nuove dipendenze senza motivarle;
- non deve dichiarare chiuso un compito senza una prova di verifica.

AGENTS.md — Claude

AGENTS.md – Claude

Ruolo

Progettista, revisore e chiarificatore dei rischi.

Obiettivi

- chiarire i punti ambigui;
- preparare piani di realizzazione;
- fare una revisione critica del lavoro altrui;
- far emergere rischi, possibili errori e problemi lasciati in sospeso.

Che cosa deve ricevere prima di iniziare

- la specifica;
- le decisioni tecniche registrate;
- il compito;
- il risultato prodotto da chi ha realizzato il lavoro;
- il contesto tecnico generale.

Che cosa deve restituire

- una sintesi del problema;
- le assunzioni fatte;
- i rischi individuati;
- i compromessi presi in considerazione;
- un giudizio esplicito se si può procedere oppure no.

Cosa non deve mai fare

- non deve sostituire una prova reale con un discorso convincente ma vuoto;
- non deve approvare un lavoro che non si può tracciare con chiarezza;
- non deve trascurare i rischi solo perché la soluzione appare elegante.

AGENTS.md — Perplexity

AGENTS.md – Perplexity

Ruolo

Chi mette alla prova con ricerche esterne e verifica le fonti.

Obiettivi

- cercare fonti esterne affidabili;
- proporre alternative;
- contestare assunzioni deboli;
- confrontare standard, schemi e prassi già diffuse.

Che cosa deve ricevere prima di iniziare

- l'affermazione da verificare;
- le domande di confronto;
- eventuali limiti di tempo o di area geografica;
- i criteri di qualità richiesti alla fonte.

Che cosa deve restituire

- fonti collegate con un link;
- un confronto sintetico;
- i punti su cui non è d'accordo;
- i limiti delle prove trovate.

Cosa non deve mai fare

- non deve produrre codice del progetto come compito principale;
- non deve trasformare un confronto generico in una regola assoluta;
- non deve dichiarare una certezza dove esistono solo indizi.

AGENTS.md — DeepSeek Coder

AGENTS.md – DeepSeek Coder

Ruolo

Agente di scrittura del codice attento ai costi.

Obiettivi

- assorbire compiti tecnici ripetitivi;
- produrre riscritture meccaniche;
- estendere l'impalcatura dei collaudi;
- gestire modifiche ripetute su tanti file, a costo contenuto.

Che cosa deve ricevere prima di iniziare

- i file da modificare;
- lo schema da applicare;
- i vincoli di stile;
- i limiti del proprio perimetro di lavoro.

Che cosa deve restituire

- la modifica proposta;
- una nota sui punti dubbi incontrati;
- l'elenco delle parti da far rivedere a un revisore principale.

Cosa non deve mai fare

- non deve prendersi la responsabilità di decisioni architettoniche;
- non deve agire su parti delicate del sistema senza una doppia revisione;
- non deve dare per corrette assunzioni di settore mai verificate davvero.

AGENTS.md — Coordinatore del gruppo di agenti

AGENTS.md – Coordinatore del gruppo di agenti

Ruolo

Coordina il lavoro di più agenti insieme, con ruoli diversi.

Obiettivi

- assegnare il compito al ruolo giusto;
- imporre un ordine chiaro di intervento;
- raccogliere i pareri contrari o discordanti;
- chiudere con una decisione, un punto di controllo e il prossimo passo.

Ordine obbligatorio da seguire

1. fatti;
2. assunzioni;
3. rischi;
4. alternative;
5. decisione.

Che cosa deve restituire

- la decisione presa;
- quali agenti sono stati coinvolti;
- quali documenti sono stati letti;
- i rischi rimasti aperti;
- a partire da quale segnale bisogna far intervenire una persona.

Cosa non deve mai fare

- non deve confondere il governo dell'archivio con il lavoro degli specialisti di settore;
- non deve chiudere in positivo senza un vero confronto critico;
- non deve nascondere i punti di disaccordo emersi.

AGENTS.md — Agente specialista di settore

AGENTS.md – Agente specialista di settore

Ruolo

Agente specialistico di un settore, su una piattaforma dedicata come Regolo.ai o equivalente.

Obiettivi

- verificare le regole del proprio settore;
- segnalare le eccezioni;
- proporre casi limite da considerare;
- distinguere con chiarezza un fatto normativo, una prassi diffusa e una semplice ipotesi.

Che cosa deve ricevere prima di iniziare

- la specifica di settore;
- il glossario dei termini del settore;
- il caso concreto da valutare;
- una domanda ben delimitata.

Che cosa deve restituire

- la risposta specialistica;
- le regole applicate;
- i casi rimasti dubbi;
- le parti che vanno comunque affidate a una revisione umana.

Cosa non deve mai fare

- non deve riscrivere il governo generale dell'archivio di progetto;
- non deve prendere decisioni fuori dal proprio settore di competenza;
- non deve nascondere incertezze normative o interpretative.

Capitolo 18 — Alcune riflessioni ulteriori maturate nel tempo

18.1 Gli esempi vanno letti come strumenti di ragionamento

Gli esempi pratici del Capitolo 16 hanno valore soprattutto quando vengono letti come schemi di scomposizione di un problema, non come soluzioni già pronte da copiare: ogni esempio dovrebbe rendere chiaro: il confine del problema; il livello di rischio; la separazione tra chi governa, chi è specialista e chi verifica; il punto in cui il giudizio di una persona resta comunque indispensabile. Lo scopo di quegli esempi è mostrare come lo stesso metodo regga su argomenti molto diversi tra loro.

18.2 La tabella ruolo-strumento è un documento centrale

La tabella che mette in relazione strumento, ruolo, ciò che riceve, ciò che restituisce, i limiti e dove opera (Capitolo 15) è probabilmente il documento di governo più importante, perché permette di: evitare che due ruoli si sovrappongano; rendere sostituibili gli strumenti e i fornitori tecnologici senza dover ripensare tutto; evitare di trattare un modello di intelligenza artificiale come se fosse una persona con intenzioni proprie; distinguere con chiarezza che cosa uno strumento sa fare, quali sono i suoi limiti e in quale contesto va usato.

Questa tabella va aggiornata ogni volta che cambia almeno uno di questi elementi: il perimetro del compito; quanto sono delicati i dati coinvolti; il costo che ci si può permettere; la velocità di risposta richiesta; il livello di autonomia concesso.

18.3 I file dei ruoli funzionano come accordi operativi

I file di tipo **AGENTS.md** funzionano bene quando restano brevi, verificabili e centrati sul comportamento concreto atteso. Le parti minime indispensabili sono: il ruolo; gli obiettivi; che cosa deve ricevere prima di iniziare; che cosa deve restituire; che cosa non deve mai fare; le condizioni che richiedono di far intervenire una persona.

Divieti scritti con chiarezza tengono l'agente dentro il proprio perimetro, un elenco chiaro di ciò che deve ricevere prima di iniziare impedisce che il contesto venga "riempito per supposizione" e un elenco chiaro di ciò che deve restituire rende molto più facile verificare la qualità del lavoro.

18.4 Il governo dell'archivio e il livello specialistico devono restare separati

I problemi nascono soprattutto quando: chi governa l'archivio prende decisioni di settore senza consultare gli specialisti; uno specialista di settore detta lui stesso le regole di governo dell'archivio; una ricerca esterna viene scambiata per una verifica interna già fatta; lo strumento più economico viene usato fuori dal proprio ambito solo per risparmiare.

La regola che ne deriva: chi governa l'archivio si occupa del processo, dell'archivio stesso, dei compiti, della revisione e dello smistamento del lavoro; il livello specialistico si occupa dei contenuti di settore, delle regole del settore, delle eccezioni e dei casi limite; la validazione finale resta sempre in capo a chi si assume la responsabilità del rischio di quel pezzo di lavoro.

18.5 Un linguaggio impersonale aiuta la stabilità del documento

Scrivere in modo impersonale, evitando riferimenti diretti a singole persone, è una scelta di governo del documento, non solo di stile: un testo impersonale riduce la dipendenza da singole persone o nomi propri; aumenta la possibilità di riusare e diffondere il documento; evita che il metodo venga scambiato per un'opinione personale di qualcuno; rende più facile usare il documento come riferimento condiviso da più gruppi di lavoro.

Per questo motivo la forma impersonale è particolarmente indicata nelle parti che descrivono: i ruoli degli agenti; le regole di smistamento del lavoro; i criteri di qualità; le condizioni che impongono di fermarsi o di far intervenire una persona.

18.6 Alcuni errori da evitare nella scrittura della documentazione

- esempi presentati come racconti generici, senza una scomposizione reale in compiti verificabili;
- tabelle troppo astratte, senza limiti né vincoli concreti;
- file **AGENTS.md** che spiegano solo che cos'è un modello, senza regolarne davvero il comportamento;
- dare per scontata una tecnologia o un'infrastruttura non realmente disponibile;
- confondere "consapevole dei limiti di un fornitore" con "vincolato per sempre a un solo fornitore";

- approvare un documento senza un vero contraddittorio tra chi propone e chi mette alla prova.

18.7 Criteri di qualità per chi aggiorna questo documento in futuro

Ogni futura aggiunta a questa dispensa dovrebbe essere valutata da un piccolo gruppo con quattro funzioni distinte:

- **chi progetta** controlla la struttura, i confini e la coerenza degli schemi usati;
- **chi realizza** trasforma le richieste in sezioni concrete e tabelle verificabili;
- **chi rivede** segnala dimenticanze, ripetizioni inutili, ambiguità e argomenti fuori tema;
- **chi mette alla prova** contesta le assunzioni, i favoritismi verso un fornitore e le affermazioni non dimostrate.

Una nuova sezione si può considerare accettabile quando rispetta almeno questi criteri: è riusabile anche fuori da un contesto specifico; distingue chiaramente un fatto, uno schema generale e un consiglio; definisce un risultato verificabile; segnala almeno un rischio o un limite; non richiede di conoscere già tecnologie non disponibili a chi legge.

Capitolo 19 — Che cosa fare nella prima settimana

La prima settimana serve soprattutto a capire se il metodo sta prendendo una forma sana. Conviene scegliere un modulo pilota piccolo e ben delimitato, scrivere in una sola pagina la costituzione del progetto, preparare una specifica vera con criteri di accettazione chiari, e mettere al lavoro due agenti sullo stesso compito ma con ruoli diversi.

A quel punto conta un solo dato: se il metodo sta migliorando davvero chiarezza, qualità e tempo impiegato. Tutto il resto viene dopo.

Capitolo 20 — La direzione consigliata, in sintesi

La direzione più solida consiste nel partire con un piccolo "sistema operativo per il metodo SDD", fatto di file, regole, ruoli e verifiche. **Codex**, **Claude**, **DeepSeek**, **Coder** e **Perplexity** trovano posto nel livello di governo dell'archivio di progetto; **Regolo.ai** presidia invece il livello specialistico dei settori, delle applicazioni verticali e dei modelli legati al business.

Groq e fornitori simili si inseriscono bene come capacità economica da usare quando serve, mentre **Runpod** e soluzioni simili rappresentano una direzione futura per la libertà di scelta del fornitore e il controllo dei costi.

Gruppi di agenti più ampi, agenti dedicati, collegamenti a strumenti esterni e automazioni più spinte conviene introdurli solo dopo aver messo alla prova il metodo su un progetto pilota reale.

Fonti consultate e verificate

- Repository ufficiale di GitHub Spec Kit: github.com/github/spec-kit
- Documentazione di GitHub Spec Kit: github.github.com/spec-kit/
- Blog ufficiale GitHub sullo Sviluppo Guidato dalla Specifica: github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-

[toolkit/](#)

- Martin Fowler, *Specification by Example*: martinfowler.com/bliki/SpecificationByExample.html
- Martin Fowler, *Given When Then*: martinfowler.com/bliki/GivenWhenThen.html
- Cucumber, *Behavior-Driven Development*: cucumber.io/docs/bdd/
- Voce Wikipedia, *Test-driven development*: en.wikipedia.org/wiki/Test-driven_development
- Anthropic, *Building effective agents*: anthropic.com/research/building-effective-agents
- Anthropic, guida al prompt engineering per Claude: docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview
- OpenAI, guida al prompt engineering: developers.openai.com/api/docs/guides/prompt-engineering
- VS Code, panoramica della chat con l'intelligenza artificiale: code.visualstudio.com/docs/chat/chat-overview
- Model Context Protocol, introduzione: modelcontextprotocol.io/introduction
- de Bono Group, *Six Thinking Hats*: debonogroup.com/services/core-programs/six-thinking-hats/

Chiusura

La versione più solida di questo metodo, per chi parte da zero, è quella che resta leggibile, verificabile e disciplinata anche quando il progetto cresce di dimensioni.

Se il lavoro comincia da buone specifiche, istruzioni riusabili, ruoli semplici, un vero confronto critico tra chi propone e chi mette alla prova, collaudi automatici e decisioni sempre registrate, allora diventa naturale, con il tempo, evolvere verso gruppi di agenti più ampi, automazione e maggiore autonomia — partendo però sempre da basi già solide.

by Fabrizio Ricciarelli, giugno 2026